Lab2：BombLab 实验报告

唐泽楠　24307140056

2026 年 10 月 12 日

1　概览

六个正式关卡与 Honor 全部完成。`python3 public_test.py` 结果为 `Public tests: 60/60`、`Secret public tests: 10/10`；另外用 seed `24307140056` 完整运行，从第 1 关一直通到 `Secret phase defused.`。

分析方法。　先用 `objdump -d --no-show-raw-insn bomb` 导出整份反汇编离线阅读，找到 `phase_01`～`phase_06` 和它们调用的辅助函数（`p1_step*`、`p3_case_*`、`p4_walk`、`p5_traverse`、`p6_op_*` 等），再从 `.rodata` 读出跳转表、链表节点和函数指针表。推出公式后，在 GDB 中逐关停在比较指令处，把寄存器里的期望值、输入值和公式算出的值对照起来。

几处共同点：

- 每关入口 `rdi` 是输入字符串，`rsi` 是 64 位 challenge（下文记作 c）。第一步都调用 `valid_u32_fields(input, n)` 检查输入是否恰好是 n 个 32 位无符号整数，再调用 `sscanf`。格式串在 `.rodata` 中：`0x403182="%u"`，`0x40317f="%u %u"`，`0x40317c="%u %u %u"`，`0x403173=` 六个 `%u`。所以 n 就是口令中整数的个数。
- 记 $\mathrm{lo}(x)$ 为 x 的低 32 位，$\mathrm{hi}(c) = \mathrm{lo}(c \gg 32)$。除特别说明外，所有运算都是 32 位无符号运算（模 2^{32}），在 C 中统一用 `uint32_t`。

2　测试结果截图

```
nanny@ZenandeMacBook-Pro ~ % docker exec -it -w /work/ics/ics-2026-bomblab ics-lab bash -c 'uname -sm; make test 2>&1 |
tail -14'
Linux x86_64
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

图 1: 公开测试：正式六关 60/60，Honor 10/10（`make test` 输出末尾）

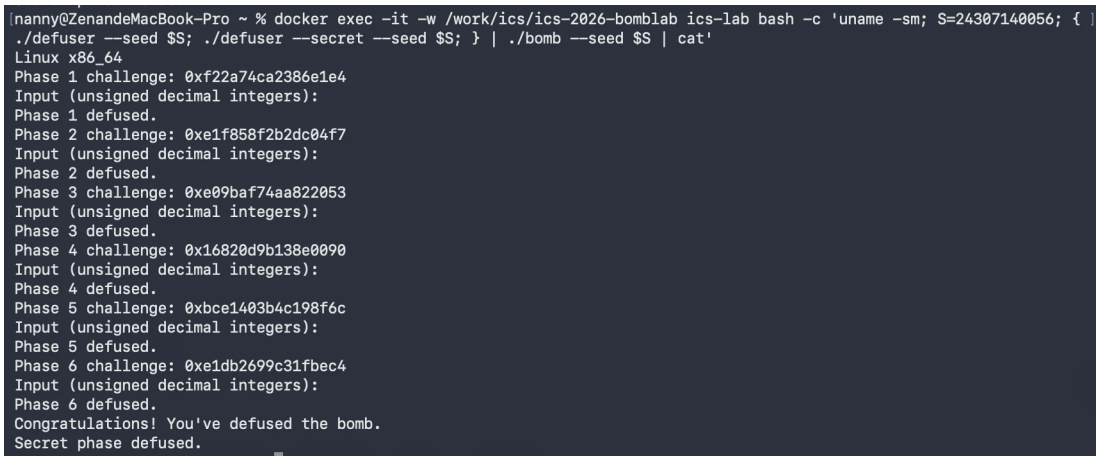


图 2: seed 24307140056：六关 + Honor 全部通过

3 第 1 关：函数调用与参数传递

关键汇编

```
phase_01:   ... valid_u32_fields(input, 1); sscanf(input, "%u", -0x14(%rbp))
  402068: mov  %r12,%rdi              # r12 = challenge
  40206b: call p1_compute
  402070: cmp  %eax,-0x14(%rbp)       # 返回值 == 输入的唯一整数 ?
p1_compute:
  401fe5: mov  %rdi,%rdx              # rdx = c
  401fe8: mov  %rdi,%rax
  401feb: shr  $0x20,%rax             # rax = c >> 32
  401fef: mov  %rdx,%rsi
  401ff2: mov  %eax,%edi              # edi = hi(c)
  401ff4: shr  $0x10,%rsi             # esi = lo(c >> 16)
  401ffb: call p1_step1               # t1 = p1_step1(hi, lo(c>>16))
  402000: mov  %edi,%esi              # esi = hi(c)   (step1 没改 edi/edx)
  402002: mov  %edx,%edi              # edi = lo(c)
  402004: mov  %eax,%ecx              # ecx = t1
  402006: call p1_step0               # t0 = p1_step0(lo, hi)
  40200b: mov  %ecx,%esi
  40200d: mov  %eax,%edi
  40200f: call p1_step2               # return p1_step2(t0, t1)
p1_step0:  lea (%rdi,%rdi,4),%eax ; add %esi,%eax ; xor $0x7866,%eax
p1_step1:  lea 0x2317(%rsi),%eax  ; xor %edi,%eax
p1_step2:  sub %esi,%edi ; lea 0x69a1(%rdi,%rdi,2),%eax
```

推理与计算关系

`p1_step1` 只读 `edi`、`esi`，写 `eax`，所以调用之后 `edi` 仍是 $\mathrm{hi}(c)$，`edx` 仍是 c。这正是 402000 处能直接把它们拿来当下一次调用参数的原因。三个小函数都用 `lea` 完成乘加：`(%rdi,%rdi,4)`

是 $5x$，`0x69a1(%rdi,%rdi,2)` 是 $3x + \texttt{0x69a1}$。

$$t_1 = (\mathrm{lo}(c \gg 16) + \texttt{0x2317}) \oplus \mathrm{hi}(c),$$
$$t_0 = (5 \cdot \mathrm{lo}(c) + \mathrm{hi}(c)) \oplus \texttt{0x7866},$$
$$\text{答案} = 3\,(t_0 - t_1) + \texttt{0x69a1}.$$

口令只有 1 个整数。

4 第 2 关：数组、循环与递推

关键汇编

```
4020de: call __isoc99_sscanf       # "%u %u %u %u %u %u" -> a[0..5] 位于 -0x40(%rbp) 起
4020e5: cmp  $0x6,%eax
4020ea: mov  %r13,%rax              # r13 = c
4020ed: mov  -0x40(%rbp),%esi       # esi = a[0]
4020f0: shr  $0x10,%rax
4020f4: and  $0x3f,%eax
4020f7: xor  $0x1,%eax
4020fa: cmp  %eax,%esi              # a[0] == ((c>>16)&63) ^ 1
4020fe: mov  $0x1,%edx              # i = 1
402103: lea  0xd(%rdx),%ecx         # cl = i + 13
402106: mov  %r13,%rax
402109: shr  %cl,%rax
40210c: and  $0x3f,%eax             # (c >> (i+13)) & 63
40210f: add  %edx,%eax              #   + i
402111: xor  %esi,%eax              #   ^ a[i-1]
402113: mov  -0x40(%rbp,%rdx,4),%esi   # esi = a[i]
402117: lea  (%rax,%rax,2),%eax     #   * 3
40211a: cmp  %eax,%esi
40211e: add  $0x1,%rdx
402122: cmp  $0x6,%rdx
402126: jne  402103
```

推理与计算关系

`sscanf` 的 6 个输出地址是 `-0x40,-0x3c,…,-0x2c(%rbp)`，相邻差 4 字节，所以这是一个 `uint32_t a[6]`。循环里用 `-0x40(%rbp,%rdx,4)` 按下标取元素。`esi` 在每轮末尾被换成 $a[i]$，下一轮再拿去异或，因此它在下一轮中就是 $a[i-1]$。由此恢复出递推式：

$$a_0 = ((c \gg 16)\,\&\,\texttt{0x3f}) \oplus 1, \qquad a_i = 3\big(\,(((c \gg (13+i))\,\&\,\texttt{0x3f}) + i) \oplus a_{i-1}\big), \quad i = 1, \ldots, 5.$$

口令是 $a_0, \ldots, a_5$ 共 6 个整数。注意移位量 $13 + i$ 作用在 64 位的 c 上（`shr %cl,%rax`），乘法和加法是 32 位的。

5 第 3 关：switch 与跳转表

关键汇编

```
phase_03:  ... sscanf(input, "%u %u", -0x18(%rbp), -0x14(%rbp))
  40228c: mov  %r12d,%edi
  402291: and  $0x7,%edi
  402294: cmp  %edi,-0x18(%rbp)       # 第一个数 == c & 7
  402299: mov  %r12,%rax
  40229c: mov  %r12,%rsi
  40229f: shr  $0x3,%rax
  4022a3: shr  $0x20,%rsi
  4022a7: xor  %eax,%esi              # esi = lo(c>>3) ^ hi(c)
  4022a9: call p3_dispatch            # (edi = c&7, esi = x)
  4022ae: cmp  %eax,-0x14(%rbp)       # 第二个数 == 分支结果
p3_dispatch:
  4021e4: mov  %edi,%eax
  4021e6: mov  %esi,%edi              # 分支函数的参数是 x
  4021e8: cmp  $0x7,%eax
  4021eb: ja   402235                 # 越界 -> 返回 0
  4021ed: notrack jmp *0x4057e0(,%rax,8)
```

跳转表与各分支

从 0x4057e0 读出 8 个 8 字节表项，每项指向 p3_dispatch 中的一条 jmp p3_case_N（尾调用）。注意跳转表下标和 p3_case_N 的编号并不一致：

下标 $c\&7$	表项地址	跳到	分支函数（参数 x）
0	0x402208	p3_case_4	lea 0x2ba0(%rdi,%rdi,2) $\Rightarrow 3x + 0\text{x}2\text{ba}0$
1	0x402210	p3_case_2	$(x \oplus 0\text{x}4\text{d}29) + (x \gg 1)$
2	0x402218	p3_case_6	$(x \oplus 0\text{xe}534) + (x \gg 2)$
3	0x402220	p3_case_1	$4x \oplus 0\text{xe}841$
4	0x402228	p3_case_5	$4x \oplus 0\text{xb}6\text{b}7$
5	0x402230	p3_case_3	$3x + 0\text{xff}0\text{e}$
6	0x4021f8	p3_case_7	$5x + 0\text{x}4\text{e}29\text{b}$
7	0x402200	p3_case_0	$5x + 0\text{x}3594\text{e}$

计算关系

口令为 2 个整数：第一个是 $k = c\&7$；第二个是 $f_k(x)$，其中 $x = \mathrm{lo}(c \gg 3) \oplus \mathrm{hi}(c)$，$f_k$ 按上表选取。由于 $k \le 7$，ja 的越界分支不会走到。解法中用 p3_jump_table[8] = {4,2,6,1,5,3,7,0} 记录“表项 → case 编号”，再用 switch 实现 8 个分支。

6 第 4 关：递归

关键汇编

```
phase_04:  ... sscanf(input, "%u", -0x14(%rbp))
  402378: mov  %r12,%rcx ; mov %r12,%rax ; mov %r12d,%edi
  402381: xor  %esi,%esi              # lo  = 0
  402383: shr  $0x20,%rax
  402387: shr  $0x7,%rcx
  40238b: and  $0x7f,%edi             # target = c & 127
  40238e: mov  $0x7f,%edx             # hi  = 127
  402393: xor  %eax,%ecx              # key = lo(c>>7) ^ hi(c)
  402395: call p4_walk
  40239a: cmp  %eax,-0x14(%rbp)
p4_walk(edi=target, esi=lo, edx=hi, ecx=key):
  4022c4: mov  %edi,%eax              # eax = target
  4022c6: mov  %edx,%edi
  4022c8: sub  %esi,%edi
  4022ca: shr  $1,%edi
  4022cc: add  %esi,%edi              # mid = lo + (hi-lo)/2
  4022ce: cmp  %eax,%edi
  4022d0: je   402320                 # 找到: return (mid+0xc4) ^ key
  4022d7: lea  (%rdi,%rcx,1),%ebx     # ebx = mid + key (被调用者保存寄存器)
  4022de: cmp  %edi,%eax
  4022e0: jb   402300                 # target < mid -> 左半
  4022e2: lea  0x1(%rdi),%esi         # 右半: lo = mid+1
  4022e5: mov  %eax,%edi
  4022e7: call p4_walk
  4022ec: lea  (%rax,%rax,2),%eax
  4022ef: xor  %ebx,%eax
  4022f6: xor  $0x4,%eax              # return (3r ^ (mid+key)) ^ 4
  402300: lea  -0x1(%rdi),%edx        # 左半: hi = mid-1
  402303: mov  %eax,%edi
  402305: call p4_walk
  40230a: lea  (%rax,%rax,2),%eax
  40230d: xor  %ebx,%eax
  402314: xor  $0x17,%eax             # return (3r ^ (mid+key)) ^ 0x17
  402320: lea  0xc4(%rdi),%eax
  402326: xor  %ecx,%eax
```

推理与计算关系

`p4_walk` 是在 $[0,127]$ 上对 `target` 做二分查找。`mid` 用 `lo + ((hi-lo)>>1)` 计算（无符号右移）；`rcx`（key）在递归调用中原样传下去；`rbx` 是被调用者保存寄存器，所以函数开头先 `push %rbx` 保存，再用它存放本层的 `mid+key`，等子调用返回后继续使用。递归式为

$$W(t,l,h)=\begin{cases}(m+\texttt{0xc4})\oplus k, & m=t,\\ \big(3\,W(t,l,m-1)\oplus(m+k)\big)\oplus\texttt{0x17}, & t<m,\\ \big(3\,W(t,m+1,h)\oplus(m+k)\big)\oplus\texttt{0x4}, & t>m,\end{cases}\qquad m=l+\lfloor(h-l)/2\rfloor.$$

答案为 $W(c\,\&\,127,\ 0,\ 127)$，$k=\mathrm{lo}(c\gg 7)\oplus\mathrm{hi}(c)$，口令 1 个整数。由于 $t\in[0,127]$ 一定会被找到，递归深度不超过 7 层；左半分支时 $m\geq 1$，所以 $m-1$ 不会回绕。“返回过程”的要点是：结

果从最深一层（命中的那层）开始，在逐层返回时计算，每层把子调用的返回值乘 3、异或本层的 $m+k$，再按本层走的方向异或 0x17（左）或 0x4（右）。

7 第 5 关：结构体与链表遍历

关键汇编

```
phase_05:  ... sscanf(input, "%u %u", -0x1c(%rbp), -0x18(%rbp))
  40245c: lea  -0x14(%rbp),%rsi       # 输出参数 &last
  402463: call p5_traverse            # eax = acc
  40246c: cmp  %edx,-0x1c(%rbp)       # 第一个数 == acc
  402474: cmp  %eax,-0x18(%rbp)       # 第二个数 == last
p5_traverse(rdi=c, rsi=&last):
  4023bd: and  $0x7,%eax
  4023c0: shr  $0x3,%rcx              # rcx = c >> 3
  4023c4: mov  0x405b20(,%rax,8),%rdx # node = heads[c & 7]
  4023cf: shr  $0x20,%rdi
  4023d3: shr  $0x6,%rax
  4023d7: mov  0x8(%rdx),%esi         # node->value
  4023da: xor  %edi,%eax
  4023dc: xor  0xc(%rdx),%eax         #   ^ node->key
  4023e1: xor  $0x8b6f,%eax
  4023e6: mov  %esi,(%r8)             # *last = node->value
  4023e9: add  %esi,%eax              # acc = (lo(c>>6)^hi(c)^key^0x8b6f) + value
  4023eb: and  $0x3,%edi              # steps = (c>>3) & 3
  4023ee: je   40240a
  4023f0: mov  $0x1,%ecx
  4023f5: mov  (%rdx),%rdx            # node = node->next
  4023f8: add  $0x1,%ecx
  4023fb: mov  0x8(%rdx),%esi
  4023fe: xor  0xc(%rdx),%eax         # acc ^= node->key
  402401: add  %esi,%eax              # acc += node->value
  402403: mov  %esi,(%r8)             # *last = node->value
  402406: cmp  %ecx,%edi
  402408: jae  4023f5                 # 共走 steps 步
```

结构体布局与链表数据

从访问方式 (%rdx)、0x8(%rdx)、0xc(%rdx) 可知节点结构为

```
struct node {
    struct node *next;   /* +0x0 */
    uint32_t     value;  /* +0x8 */
    uint32_t     key;    /* +0xc */
};
```

0x405b20 处是 8 个节点指针（GDB 中符号名为 p5_heads）。顺着 next 读出整条链，共 12 个节点，最后一个的 next 为 NULL，而 heads[i] 恰好指向链上第 i 个节点：

链上序号	0	1	2	3	4	5	6	7	8	9	10	11
地址 (0x405…)	c10	b70	bd0	b60	be0	bf0	b90	bb0	c00	bc0	b80	ba0
value	47a	856	037	36c	12b	b7b	60b	a8a	713	913	219	530
key	f12c	5cbb	8afc	1290	7a9e	2550	ab7f	5e22	78e8	7401	9ac1	5d83

（value、key 均为十六进制。）可见节点在内存中并不是按链表顺序存放的，必须顺着 `next` 走。

计算关系

设起点 $p = c\,\&\,7$，步数 $s = (c \gg 3)\,\&\,3$（循环条件是 `ecx` 从 1 到 s，共 s 步）：

$$acc = (\mathrm{lo}(c \gg 6) \oplus \mathrm{hi}(c) \oplus key_p \oplus \texttt{0x8b6f}) + value_p, \qquad acc \leftarrow (acc \oplus key_{p+j}) + value_{p+j},\ j = 1..s.$$

口令 2 个整数：acc 和最后到达节点的 $value_{p+s}$。由于 $p + s \le 10 < 12$，不会走到 `NULL`。解法中把链按顺序存成数组 `p5_chain[12]`，“走一步”就是下标加 1。

8 第 6 关：函数指针表与间接调用

关键汇编

```
phase_06:  ... sscanf(input, "%u", -0x14(%rbp)); eax = p6_dispatch(c); cmp %eax,-0x14(%rbp)
p6_dispatch:
  402509: mov  %rdi,%rbx
  40250c: shr  $0x6,%rdi
  402510: mov  %rbx,%rax
  402513: shr  $0x20,%rax
  402517: xor  %eax,%edi              # x = lo(c>>6) ^ hi(c)
  402519: mov  %rbx,%rax
  402520: and  $0x38,%ebx             # rbx = c & 0x38 = 8*((c>>3)&7)，字节偏移
  402523: and  $0x7,%eax              # rax = c & 7，下标
  402526: call *0x405dc0(,%rax,8)     # r = ops[c&7](x)
  40252d: xor  $0x709b,%eax
  402532: mov  %eax,%edi
  402534: call *0x405dc0(%rbx)        # r = ops[(c>>3)&7](r ^ 0x709b)
  40253f: add  $0x793c,%eax           # return r + 0x793c
```

函数指针表

`0x405dc0` 处 8 个表项（GDB 中符号名 `p6_ops`）依次是 `p6_op_5`，`p6_op_3`，`p6_op_0`，`p6_op_4`，`p6_op_4`，`p6_op_1`，`p6_op_1`，`p6_op_2`（6 个函数中有两个各出现两次）。各函数：

函数	关键指令	关系
p6_op_0	lea 0x3fe(%rdi),%eax; shr $3,%edi; xor %edi,%eax	$(x + \texttt{0x3fe}) \oplus (x \gg 3)$
p6_op_1	xor $0x4051,%eax; lea (%rax,%rdi,8),%eax	$(x \oplus \texttt{0x4051}) + 8x$
p6_op_2	ror $3,%eax	$\mathrm{rotr}_{32}(x, 3)$
p6_op_3	rol $7,%eax; xor $0xc374,%eax	$\mathrm{rotl}_{32}(x, 7) \oplus \texttt{0xc374}$
p6_op_4	lea 0x0(,%rdi,8),%edx; sub %edi,%edx	$d = 7x$
	shl $7,%eax; add $0xc55,%eax; xor %edx,%eax	$((x \ll 7) + \texttt{0xc55}) \oplus 7x$
p6_op_5	shr $5,%eax; xor %edi,%eax; add $0x6aa1,%eax	$((x \gg 5) \oplus x) + \texttt{0x6aa1}$

计算关系

两次间接调用串起来：

$$\text{答案} = \mathrm{ops}\big[(c \gg 3)\,\&\,7\big]\big(\mathrm{ops}[c\,\&\,7](x) \oplus \texttt{0x709b}\big) + \texttt{0x793c}, \qquad x = \mathrm{lo}(c \gg 6) \oplus \mathrm{hi}(c).$$

第二次间接调用的写法 `*0x405dc0(%rbx)` 没有比例因子，因为 `rbx = c & 0x38` 已经是“下标 $\times 8$”。`ror/rol` 是 32 位循环移位，C 中用 `(x>>n)|(x<<(32-n))` 实现。解法中按同样的顺序声明了函数指针数组 `p6_table[8]`，口令 1 个整数。

9 Honor：secret_gate / secret_copy / secret_phase

如何进入

`main` 在六关都通过、没有指定 `--phase`（内部值为 7）时，先调用 `isatty(0)`，只有 stdin 不是终端才继续用 `getc` 读第七行（最多 255 字节，读到 `'\0'` 会直接爆炸）。随后：

```
401571: call transcript_context     # 参数: 前六行口令 (main 中保存的副本)
401582: mov  %rax,%rbx              # rbx = context
401585: call phase_challenge        # esi = 7, 得到 Honor 的 challenge
40158a: mov  %rbx,%rdx              # rdx = context
40158d: lea  -0x730(%rbp),%rdi      # rdi = 第七行
401594: mov  %rax,%rsi              # rsi = challenge
401597: call secret_gate
```

`transcript_context` 与仓库中 `challenge.c` 的同名函数一致，驱动程序会算好后传给 `solve_secret`，所以求解函数直接拿到 challenge 和 context。

secret_gate：局部结构体的相邻字段覆盖

```
4026a7: mov  $0x14,%ecx
4026bb: lea  -0xd0(%rbp),%r12       # 局部对象起点 buf
4026d0: rep stos %rax,%es:(%rdi)    # 清零 20*8 = 160 字节
4026d3: movl $0x0,(%rdi)            # 再清 4 字节, 共 164 字节
4026dc: call strlen
4026e1: sub  $0x24,%rax
4026e5: cmp  $0x7f,%rax
4026e9: jbe  402700                 # 要求 36 <= len <= 163 (无符号比较)
402700: mov  %rbx,%rsi
```

```
  402703: mov  %r12,%rdi
  402706: call secret_copy            # 逐字节复制到 '\0' 为止，不检查边界
  40270b: cmpl $0x4e45504f,-0xb0(%rbp)   # buf+32 处的 4 字节 == "OPEN" ?
  402717: mov  %r14,%rdx              # context
  40271a: mov  %r13,%rsi              # challenge
  40271d: lea  -0xac(%rbp),%rdi       # buf+36 起的字符串
  402724: call secret_phase
```

局部对象一共 164 字节，被比较的 4 字节在偏移 0xd0 − 0xb0 = 32 处，交给 secret_phase 的字符串从偏移 36 开始。由此推测它是这样的结构体（和 teaching/07_stack_copy.c 中的 struct Frame 同一类）：

```
struct {
    char     name[32];   /* +0  */
    uint32_t tag;        /* +32，必须等于 0x4e45504f */
    char     args[128];  /* +36，交给 secret_phase  */
};                       /* 32 + 4 + 128 = 164 字节 */
```

secret_copy 相当于没有边界检查的 strcpy(buf.name, line)。正常情况下 tag 是 0，永远不等于 0x4e45504f；但只要第七行比 32 字节长，多出来的部分就会依次写进 tag 和 args。0x4e45504f 按小端序从低地址起的字节是 4f 50 45 4e，即 ASCII "OPEN"。所以第七行应为：

<32 个非空字符 > OPEN <secret_phase 需要的三个整数 >

总长度满足 36 ≤ len ≤ 163，复制也停在 164 字节的对象之内，没有越出栈帧，也不涉及返回地址或 shellcode，只是在同一个结构体内部覆盖了相邻字段。

secret_phase：三个整数

```
  4025df: call valid_u32_fields       # 3 个整数
  402616: call sscanf                 # "%u %u %u" -> n0, n1, n2
  402620: mov  %r13d,%ecx
  402625: and  $0x3,%ecx              # ecx = ctx & 3
  402628: cmp  %ecx,-0x2c(%rbp)       # n0 == ctx & 3
  40262d: mov  %r12,%rsi
  402630: imul $0x4854,%ecx,%r9d      # r9 = (ctx&3)*0x4854
  402637: mov  %r13,%r8
  40263a: shr  $0x20,%rsi
  40263e: shr  $0x20,%r8
  402642: xor  %r12d,%esi             # C = hi(ch) ^ lo(ch)
  402645: xor  %r13d,%r8d             # K = hi(ctx) ^ lo(ctx)
  402648: shr  $0x10,%r13
  40264c: mov  %esi,%edi
  40264e: xor  %r13d,%r9d             # r9 ^= lo(ctx >> 16)
  402651: xor  %r8d,%edi
  402654: call secret_mix             # ror(C^K, 5) ^ 0x7175
  402659: xor  %r9d,%eax
  40265c: cmp  %eax,-0x28(%rbp)       # n1 == eax
  402661: shr  $0x5,%esi
  402664: lea  0x3921(%r8,%rcx,1),%edx
```

```
40266c: add  %esi,%edx              # K + n0 + 0x3921 + (C>>5)
40266e: xor  %edx,%eax
402670: cmp  -0x24(%rbp),%eax       # n2 == n1 ^ edx
```

记 $C = \mathrm{hi}(ch) \oplus \mathrm{lo}(ch)$，$K = \mathrm{hi}(ctx) \oplus \mathrm{lo}(ctx)$：

$$
\begin{aligned}
n_0 &= ctx \,\&\, 3,\\
n_1 &= \big(\mathrm{rotr}_{32}(C \oplus K, 5) \oplus \texttt{0x7175}\big) \oplus \big(n_0 \cdot \texttt{0x4854} \oplus \mathrm{lo}(ctx \gg 16)\big),\\
n_2 &= n_1 \oplus \big(K + n_0 + \texttt{0x3921} + (C \gg 5)\big).
\end{aligned}
$$

`solve_secret` 用 `snprintf` 输出 32 个 `A`、`"OPEN"`，再接 `"%u %u %u"`。`valid_u32_fields` 允许前导空白但不要求，所以 `OPEN` 后面直接跟数字即可。

10 验证记录

关卡	seed 24307140056 的口令
1	`1455801764`
2	`29 132 570 1647 4887 14721`
3	`3 3610199481`
4	`3267942911`
5	`1372719085 2939`
6	`3262005816`
Honor	`A{32}OPEN2 2043393328 1738525096`

用下面的命令完整运行（见第 2 节截图）：

```
S=24307140056; { ./defuser --seed $S; ./defuser --secret --seed $S; } | ./bomb --seed $S
```